LINUX DAY PRATO 2026 · TRACCIA TECNICA

RedundaNet

E se una rete di storage non avesse un'azienda alle spalle, ma solo persone che si fidano?

Storage distribuito, cifrato sul tuo dispositivo, *molto* ridondante. Lo gestisce una comunità di volontari, non un'azienda.

Tahoe-LAFS · mesh VPN Tinc · identità GPG · un repo Git come source of truth · Docker Compose · Raspberry Pi, vecchi PC, un VPS da 4 €

redundanet.com · github.com/adefilippo83/redundanet



Alessio Cappellini

Istruttore tecnico · Comune di Quarrata

- RedundaNet è un progetto di Alessandro De Filippo, sviluppato dentro il ptlug. Una rete vera, con persone vere dentro, non un esperimento sul portatile.
- Io ne sono membro e contributore: ci metto un nodo e ho messo mano al codice. Oggi ve la racconto da dentro.
- Tutto quello che vedete oggi è software libero, GPL-3.

Progetto `redundanet.com · github.com/adefilippo83/redundanet`

Avete mai letto le condizioni?

AWS Customer Agreement

«The Services [...] are provided "as is." [...] [no warranty] that any content will be secure or not otherwise lost or altered.» (§8) · «You are responsible for [...] taking appropriate action to secure, protect and backup [...] Your Content.» (§2.3) · responsabilità totale: quanto hai pagato negli ultimi 12 mesi (§9.2)

Dropbox Terms

«The Services are provided "as is."» · esclusa «any loss of use, data, business, or profits, regardless of legal theory» · responsabilità limitata a «the greater of $20 USD or 100% of any amount you've paid under your current service plan».

iCloud Terms

«It is your responsibility to maintain appropriate alternate backup of your information and data.» · «Apple does not guarantee or warrant that any content you may store or access through the Service will not be subject to inadvertent damage, corruption, loss, or removal.»

Lo SLA di Amazon S3 copre la **disponibilità**: crediti del 10, 25 o 100% della fattura del mese sotto il 99,9%, il 99,0% e il 95,0% di uptime, da richiedere con un ticket entro due cicli di fatturazione. Le parole *durability* e *data loss* non compaiono. Perdere i tuoi dati non viola lo SLA.

> Se perdono i tuoi dati, il massimo che ottieni è un credito, per il downtime, da spendere sul servizio che li ha appena persi. E devi chiederlo tu.

Da dove viene: un'idea rimasta orfana



Molto distribuita (hardware qualunque, dietro i NAT di casa) · **molto ridondante** (erasure coding) · **si entra con un processo sociale**, non con una carta di credito.

Sei principi

🔒

Nessuno può leggere i tuoi dati

La cifratura avviene sul tuo dispositivo. Chi ospita vede solo ciphertext.

💥

Qualunque nodo può morire

Erasure coding, k-of-n configurabile.

📦

Nessuna autorità centrale

Identità = la tua chiave GPG. Membership = un file in Git.

🏠

Hardware qualunque, dietro NAT

Router di casa, niente port forwarding. Una mesh VPN fa da relay.

🤝

Membri verificati, non anonimi

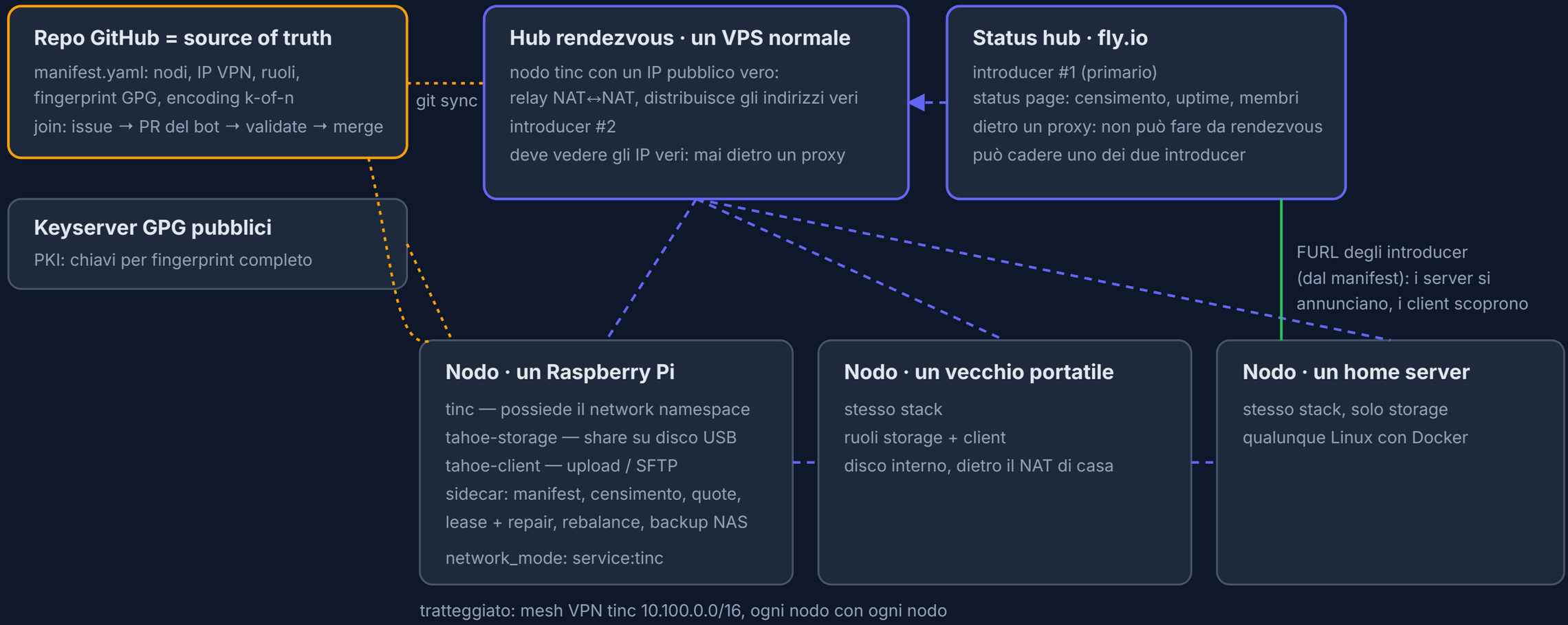
Persone approvate da persone. Alcune regole sono sulla fiducia, e lo diciamo.

🧘

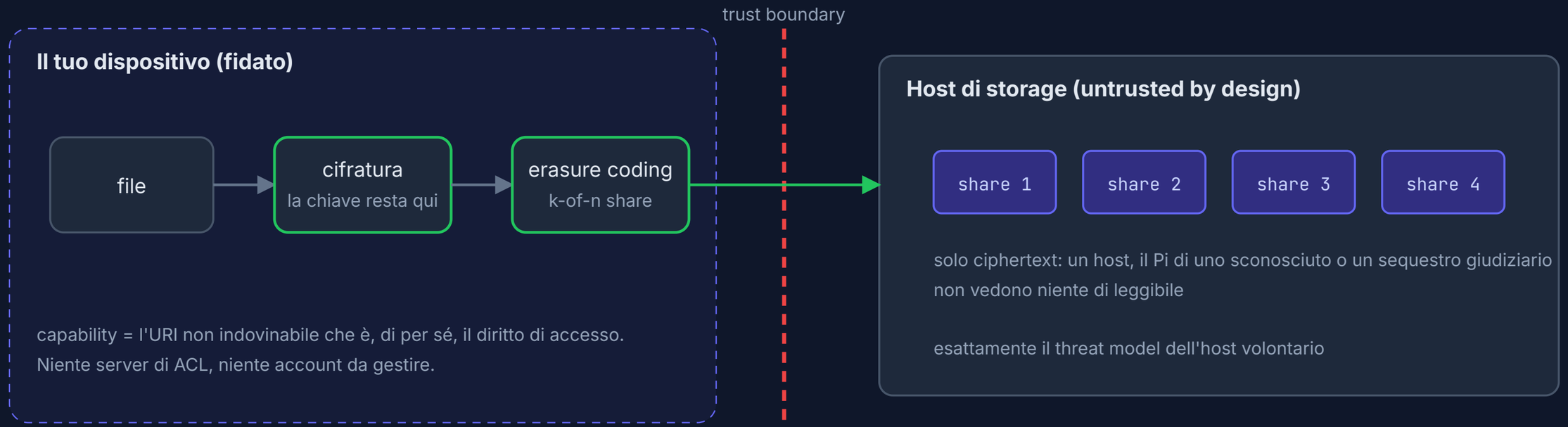
Operatività noiosa

Una CLI, uno stack compose, aggiornamento automatico con rollback, una status page.

Architettura



Perché Tahoe-LAFS: il trust boundary



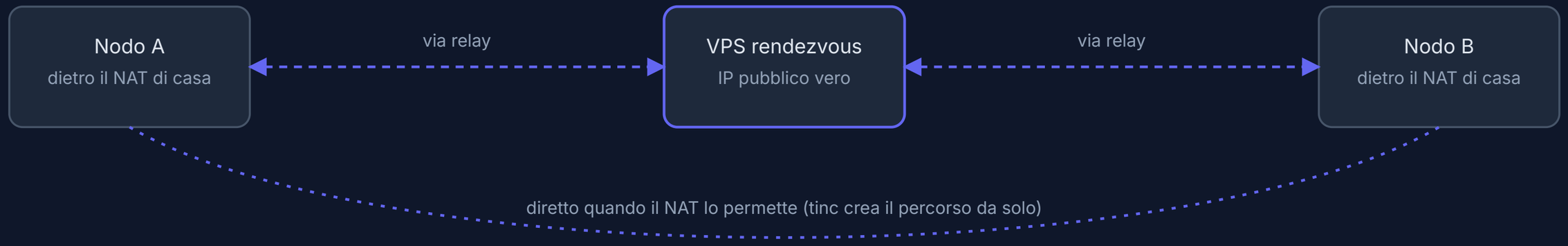
IPFS
pubblico, la persistenza dipende dalla popolarità

Ceph / MinIO
ottimi per un cluster fidato che gestisci tu

Storj / Sia
un mercato di token, non un contratto sociale

Connettività e identità: Tinc + GPG

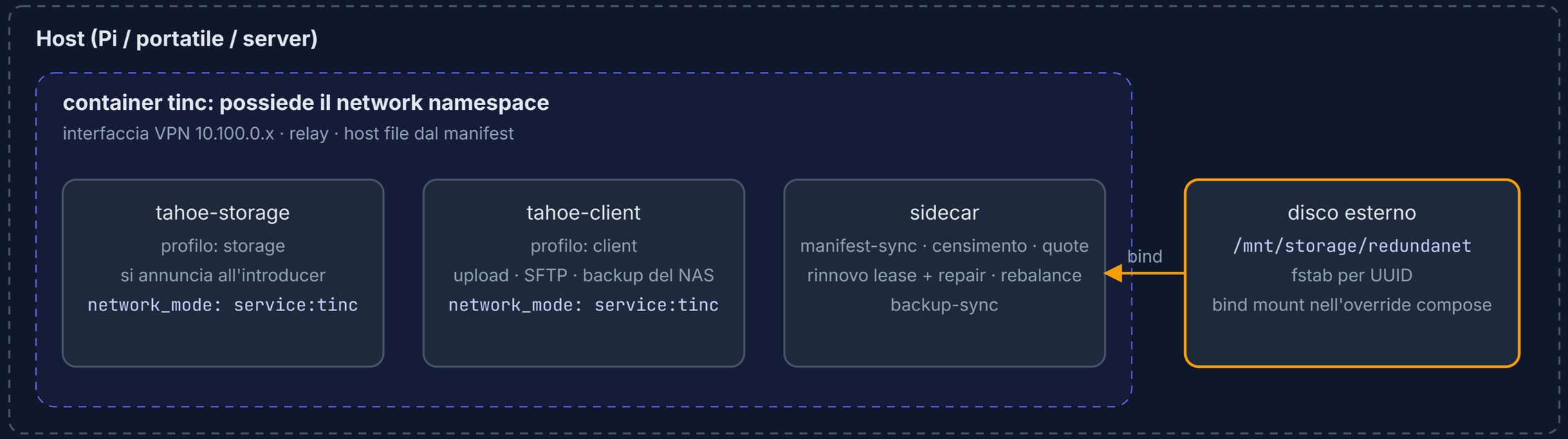
Un'identità sola: la chiave GPG del nodo è anche la sua chiave VPN



Join nella rete: fail closed



Dentro un nodo: uno stack compose, i ruoli sono profili



`docker compose -p redundanet --profile storage --profile client up -d` · i container Tahoe **entrano** nel namespace di tinc: niente port mapping da fare, e una trappola che vi faccio vedere tra poco.

LIVE DEMO 1

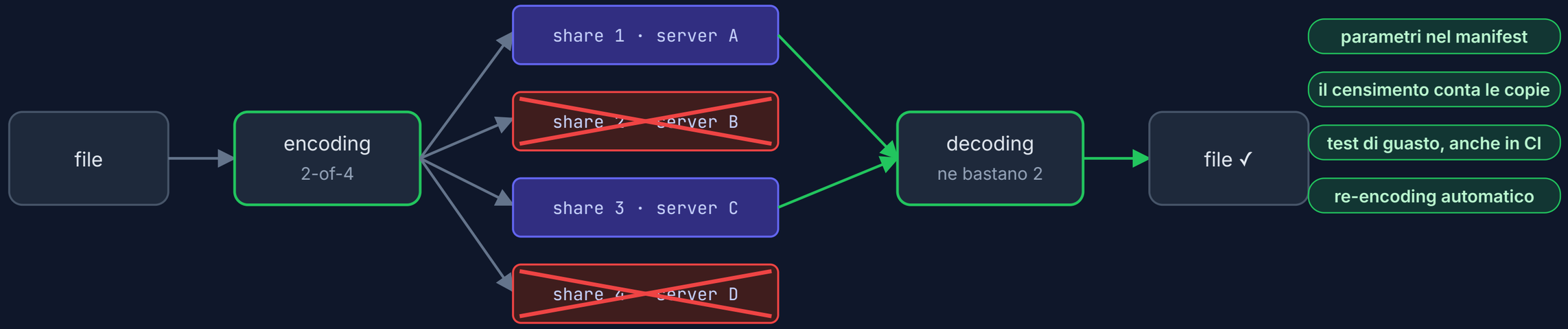
La rete, adesso

Apro `redundanet-hub.fly.dev`

- Nodi: raggiungibilità, RTT, uptime su 24h / 7d / 30d
- Server di storage annunciati agli introducer
- **Censimento della replica**: copie per oggetto rispetto al target, in continuo
- **Membri**: contributo, allocazione, utilizzo

```
# gli stessi dati della pagina (la forma; i valori li leggo dal viv
curl -s https://redundanet-hub.fly.dev/status.json \
  | jq '.replication'
{
  "objects_total": ...,
  "target_copies": 4,
  "fully_replicated": ...,
  "available_full": ...,
  "unreadable_now": 0
}
```

Ridondanza misurata, non presunta



perdi due server qualsiasi, non perdi niente · costo di storage 2×

La rete è partita come mirror **1-of-2** ed è passata a **2-of-4**: stesso costo 2×, sopravvive a due nodi. Il cambio è una riga nel manifest; ogni nodo la legge, i nuovi upload la usano subito e il rebalancer converte da solo i file vecchi.

LIVE DEMO 2

Spengo un nodo, il file resta

```
# nodo A (client): carico il file, mi segno l'hash
sha256sum talk.pdf
redundanet storage upload talk.pdf home:talk.pdf

# nodo B (storage): lo butto giù
docker stop redundanet-tahoe-storage

# status page: un server in meno, il censimento mostra disponibilità ridotta
# nodo A: il download funziona ancora, dalle copie rimaste
redundanet storage download home:talk.pdf /tmp/back.pdf
sha256sum /tmp/back.pdf     # identico

docker start redundanet-tahoe-storage   # il censimento torna a replica completa
```

DIFFICOLTÀ · INCIDENTI IN PRODUZIONE

Correggi la classe di problema, non il caso singolo



Il disco sparito

Un `-f compose.yml` esplicito disattiva l'auto-load dell'override di Docker; un update ha staccato in silenzio un disco da 1 TB. Corretto nella CLI, con un test che pretende due flag `-f`.

Il nodo nuovo che non parte

L'immagine per il Pi aveva Compose v1; un refuso nel manifest ha rotto un join per 11 minuti. Ora: Compose v2 pinnato e verificato in build, errori che dicono cosa fare al posto dei traceback.

Il rendezvous che mentiva

Un packet capture: ogni byte tra nodi passava dall'hub, in TCP. Il proxy di fly nascondeva gli indirizzi veri, e tinc si tiene il primo indirizzo che impara. Fix: un VPS da 4 € come rendezvous, una riga nel manifest.

DIFFICOLTÀ · I LIMITI DI TAHOE

Convivere con un upstream dormiente



SFTP non chiude mai le sessioni (bisect fatto: non è il client, non la rete, non Twisted) · niente accounting, quindi le quote stanno lato client

dipendenze transitive pinnate dopo che una release di pyOpenSSL 25 ha buttato giù tutti i container

La risposta

Piccoli workaround testati nelle nostre immagini · pinniamo le dipendenze che upstream non pinna · aggiriamo i frontend morti (web gateway, il pattern NAS) · documentiamo le stranezze dove l'utente le incontra.

L'assicurazione

Identità, membership e mesh non sanno che lo storage engine è Tahoe. L'engine si può sostituire; la rete no.

Hardening da un audit con 50 finding

Gate davanti alla source of truth



redundanet update



un :latest rotto non può lasciare un nodo a terra

Metodo: sei revisori per dimensione → verifica in contraddittorio di ogni finding → sei batch ordinati per blast radius, rilasciati in piccole release. Poi un secondo giro dall'interno: cosa può fare un *membro* malintenzionato? DoS sul relay, hijack degli indirizzi, storage server rogue. Classificati, in parte corretti, il resto in roadmap.

Testare un sistema distribuito su un portatile



Quello che i test **non** potevano intercettare: la trappola del netns, l'override perso, Compose v1 su un OS nuovo. Semantica dell'ambiente. Ognuno è diventato un unit test sulla costruzione dei comandi della CLI, perché il bug stava lì. E anche la build era un bug: la dipendenza Rust di Tahoe non ha wheel per ARM, quindi ogni release la compilava per un'ora sotto QEMU. Ora un'immagine base si ricostruisce solo quando cambia il lock, nativa per architettura, e una release ci mette minuti.

Usarla: un NAS di casa con una macchina del tempo erasure-coded



one-way per scelta: il sync bidirezionale vuol dire risolvere conflitti, e Magic Folder di upstream ci ha provato e ha rinunciato

l'ha proposto un membro; ha battuto l'idea iniziale con FUSE perché lavora con i file immutabili invece che contro

l'usage meter conta un file invariato una volta sola tra tutti gli snapshot; il backup si ferma se un membro supera l'allocazione

Cosa viene dopo (in ordine di rischio, non di divertimento)

Fatto: le promesse di ridondanza

- 2-of-4, convertito dal rebalancer
- due introducer, ne può cadere uno
- sweep settimanale di repair, rinnovo dei lease
- quote: allocazione per contributo, per membro
- un rendezvous che vede gli indirizzi veri
- release in minuti, non in un'ora

Adesso: hardening e pulizia

- hardening della mesh: subnet strette,
cache del censimento, meno privilegi ai container
- contributo pesato per uptime e spazio libero
- pruning degli snapshot e policy di GC
- immagine dell'hub in CI, e2e notturni, alerting
- tooling per il disco del Pi

Poi: fiducia e la storia del fork

- certificati grid-manager: solo server verificati
- accounting lato server (spending pass)
- accesso da browser via web gateway di Tahoe
- il repo come template: fork, cambi un file,
la tua rete, per esempio sui PC fermi in ufficio
- firma del manifest (rimandata apposta)

Cosa avremmo voluto sapere all'inizio

- **Fai girare la tua rete dal primo giorno.** Ogni bug serio è arrivato dalla rete in produzione, e poi è diventato un test.
- **Una validazione fuori dal percorso reale è codice morto.**
- **Scegli in base al threat model, poi isola la scelta.**
- **Dichiara solo quello che gira davvero.**
- **Rendi leggibile il fallimento:** i fix migliori sono stati messaggi d'errore che dicono cosa lanciare dopo.
- **La topologia può mentire: misura i pacchetti.**
- **Una comunità progetta meglio di te.**

Uno storage in cui si entra con una pull request.

> Molto distribuito. Molto ridondante. Nessuno in mezzo.

Codice	`github.com/adefilippo83/redundanet`
Status	`redundanet-hub.fly.dev`
Join	`redundanet.com/join.html`
Autore	`Alessandro De Filippo · ptlug · alessandro@defilippo.me`
Stack	Python · Tahoe-LAFS · Tinc · GPG · Docker · GitHub Actions · fly.io

Domande benvenute

Threat model e capability · perché non IPFS o Ceph · tinc vs WireGuard · perché un proxy non può fare da rendezvous · come funziona il censimento · cosa succede se muore un hub · quote senza accounting · cosa può fare un membro malintenzionato